

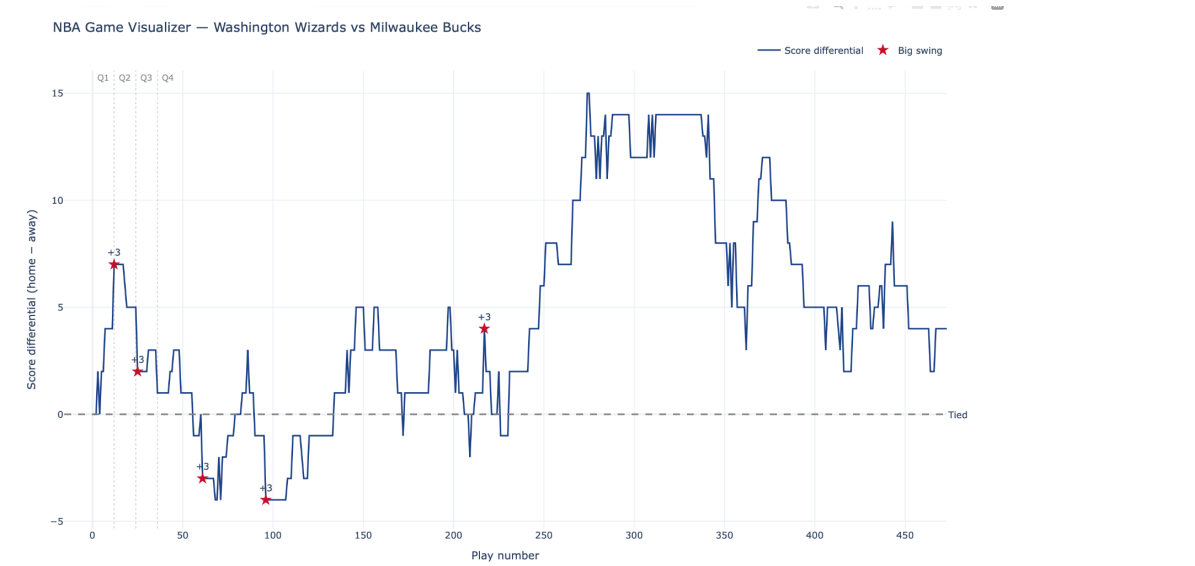
Project Overview

My project is an NBA game visualizer built using ESPN's unofficial public API. It pulls play-by-play data for a game and turns it into a running score-differential line (home score minus away score over the course of the game). With this data, it renders an interactive Plotly chart so a fan can see the shape of the game rather than just the final score. It runs as a Python script that fetches and processes the data, prints a summary to the console, and opens the finished chart in the browser.

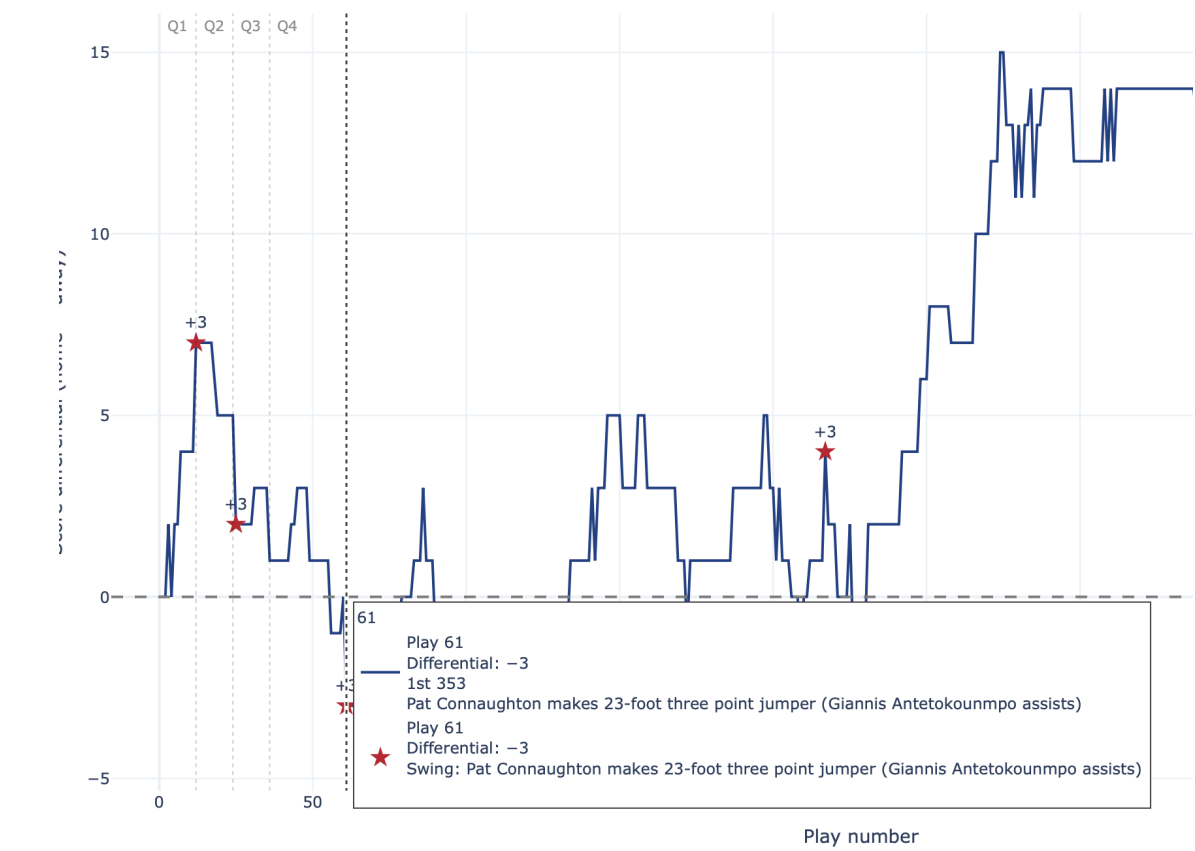
What I've Implemented So Far

- Fetch play-by-play from ESPN's summary endpoint for a given game ID, with error handling for a failed network request or invalid game ID
- Parse the play-by-play JSON into a pandas dataframe with period, clock, team, play type, play text, home/away score, and if a play was a scoring play
- Convert period and clock into an elapsed-game-time value for each play
- Compute the score differential for every play and the size of the swing from one play to the next
- Render an interactive Plotly line chart of the score differential

Demonstration



Full chart opened in browser



Close up hovering over the dataframe with play description

Issues Encountered

Game clock crashing when decimals are present:

ESPN sometimes had the game clock as a decimal string instead of a whole number. I had to change the clock-to-seconds conversion to a float() instead of int() so all values parse correctly.

Missing period value crash:

Some plays did not include a period number. The label now checks for this case and labels those plays as “unknown” instead of crashing.

Issues Not Yet Solved

Quarter divider lines plotted on the wrong scale:

I did not change the x-axis from play number to elapsed game time, so the dividers for each quarter are placed incorrectly in regards to the game. They are all clustered at the start of the chart making it look a bit confusing. This should be an easy fix I can get done for the next checkpoint.

Elapsed seconds instead of clock:

Hovering over the box shows the time value in seconds instead of a formatted game clock. Even though I understand what it should be, this would be confusing for a user.

Missing functionality of choosing or comparing a game:

The game ID is hardcoded into the file now as a placeholder. I am working on a way to choose a game as an interactive interface in the browser opening.

Where I'd Like Help

I think I want to keep version 1 as this script and clean up some of the issues. I think I will use version 2 as a stretch goal for a fuller interface with game selection and more design. I would appreciate feedback over the base functionality and these additions as well!

Milestones for the Next Few Weeks

- Fix the errors and bugs that I've run into
- Replace hard-coded place holder with user selection, can be something simple now and iterated on soon

- Test across multiple real games
- Add in documentation and update README
- Work on adding design elements

Self Reflection

Am I satisfied with my progress so far?

Yes! I've accomplished a lot more than I expected for just this first version. The core functionality is working well and I mostly have to make it more intuitive to use with customizability. I am a little behind with the interactive layer not started yet, but I feel like I've spent a good amount of time making sure everything else is working underneath that.

Can I finish the project in the remaining weeks?

I think I can realistically finish the project in the remaining weeks if I spend more time little by little working on things and committing updates early. I am also a bit less busy now that it's later in the summer so I think I will be able to dedicate more time towards the project.

Expectations vs reality

I expected the data part to be a lot more difficult, but instead I'm struggling more with figuring out interaction and design. I am also overlooking little errors here and there.

Lightbulb moments

I think in the beginning using the ESPN API was my biggest lightbulb moment as it made the data so much easier to find and work with than the other APIs I had looked up online in the project spec.